Relatório de Análise do SonarQube

Métricas de qualidade do código-fonte
Sistema de Batalhas PokeSal

A análise foi feita com o SonarQube Community 26.9, rodando localmente em um container Docker, e enviada pelo plugin do Maven (mvn clean verify sonar:sonar). A cobertura de testes foi medida pelo JaCoCo 0.8.14 durante a execução dos 192 testes JUnit e importada pelo SonarQube. A análise corresponde ao commit 49ac62e, de 26/09/2026, e os números abaixo são da visão Overall Code, que considera o projeto inteiro.

Métricas

Métrica	Resultado
Bugs	0
Vulnerabilities	0
Code Smells	47
Coverage	71,3%

Nesta versão do SonarQube, o painel não usa mais os nomes Bugs, Vulnerabilities e Code Smells: as issues aparecem agrupadas pela qualidade de software que afetam. Na captura abaixo, Reliability corresponde aos Bugs, Security às Vulnerabilities e Maintainability aos Code Smells.

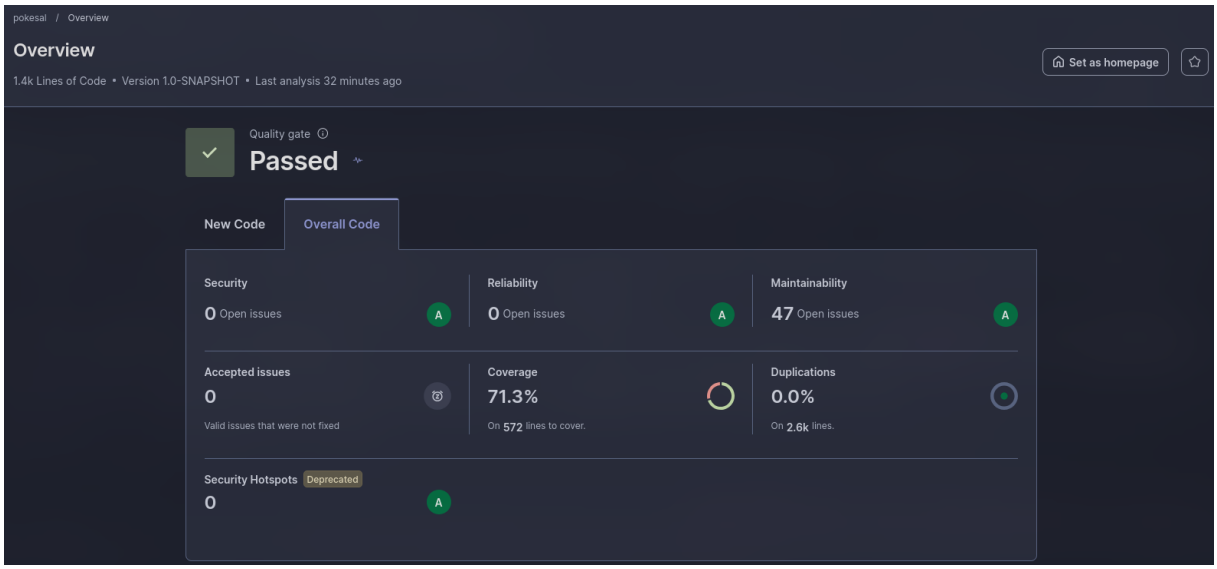


Figura 1 — Painel Overview do projeto no SonarQube, na visão Overall Code.

Bugs e Vulnerabilities

O SonarQube não apontou nenhum bug nem nenhuma vulnerabilidade. Os bugs registrados no documento de checklist estático e registro de bugs já estavam corrigidos quando a análise foi feita. Além disso, eram erros de regra de negócio (como o empate de SPD e os ganchos de passiva que não eram chamados), um tipo de problema que as regras genéricas do SonarQube não têm como detectar.

Code Smells

Os 47 code smells são todos de manutenibilidade: 2 de severidade alta, 33 média, 1 baixa e 11 informativos. Eles se dividem em oito regras, descritas abaixo com a avaliação do grupo para cada uma.

java:S106 — Uso de System.out no lugar de um logger (27 ocorrências)

Onde: BattleCli (25) e BattleRunner (2).

Avaliação: aceito. A regra parte do princípio de que System.out é usado para registrar mensagens de log, mas no PokeSal ele é a própria interface do jogo: o BattleCli existe justamente para escrever no terminal. Trocar por um logger mudaria o comportamento da aplicação.

java:S5786 — Classes de teste com modificador public (10 ocorrências)

Onde: em todas as classes de teste.

Avaliação: válido. O JUnit 5 não exige que as classes e os métodos de teste sejam públicos, então o modificador é desnecessário.

java:S5778 — Lambda de assertThrows com mais de uma chamada que pode lançar exceção (4 ocorrências)

Onde: TrainerTest (linhas 35, 113 e 131) e PokeSalTest (linha 145).

Avaliação: válido. Quando a lambda faz mais de uma chamada, o teste pode passar porque a exceção foi lançada pela chamada errada. No TrainerTest, por exemplo, o item é criado dentro da própria lambda (ItemCatalog.createBurnHeal()); o correto é criá-lo antes e deixar dentro da lambda só a chamada que deve falhar.

java:S1192 — Literais de texto repetidos (2 ocorrências)

Onde: PokeSalCatalog, com "Tackle" repetido 4 vezes e "Um ataque físico básico." repetido 6 vezes.

Avaliação: aceito. O catálogo guarda os dados de cada PokeSal de forma tabular, e o mesmo critério já tinha sido usado para suprimir a regra de números mágicos do Checkstyle nesse arquivo.

java:S107 — Construtor com mais de 7 parâmetros (1 ocorrência)

Onde: construtor do PokeSal, com 9 parâmetros.

Avaliação: válido. Os parâmetros são todos obrigatórios e validados, mas um construtor longo facilita trocar a ordem de dois valores do mesmo tipo, como ATK e DEF. Uma alternativa seria um builder.

java:S1172 — Parâmetro não usado (1 ocorrência)

Onde: parâmetro args do método main, no BattleRunner.

Avaliação: falso positivo. A assinatura main(String[] args) é exigida pelo Java para o ponto de entrada da aplicação, mesmo que os argumentos não sejam usados.

java:S7467 — Variável de exceção não usada (1 ocorrência)

Onde: bloco catch da leitura de opções no BattleCli.

Avaliação: válido. A variável e da NumberFormatException não é usada, e a partir do Java 22 ela pode ser substituída pelo padrão sem nome (_).

java:S1135 — Comentário TODO pendente (1 ocorrência)

Onde: javadoc da interface Battleground, linha 9.

Avaliação: falso positivo. Não há tarefa pendente: o texto diz que o terreno "pertence à batalha como um todo". A regra procura a palavra TODO (do inglês to do) e confundiu com o "todo" do português.

Coverage

A cobertura geral é de 71,3%, calculada pelo SonarQube combinando linhas (70,3%) e condições (74,3%). A distribuição por pacote mostra de onde vem a parte não coberta:

Pacote	Cobertura de linhas
services	100,0% (15 de 15)
entities/effects	96,6% (28 de 29)
entities/turnactions	92,6% (25 de 27)
entities	91,3% (209 de 229)
entities/passives	75,9% (41 de 54)
entities/battlegrounds	70,0% (14 de 20)
usecases	68,5% (37 de 54)
entities/statusconditions	62,5% (15 de 24)
entities/catalogs	58,1% (18 de 31)
io	0,0% (0 de 75)
raiz (BattleRunner)	0,0% (0 de 14)

O pacote io (a classe BattleCli) e o BattleRunner não são executados por nenhum teste. Os dois cuidam da interação com o jogador pelo terminal, lendo o teclado com Scanner e escrevendo com System.out, e somam 89 linhas sem cobertura. O mesmo vale para os casos

de uso ChooseInitialPokeSalUseCase e ChooseItemsUseCase, que dependem do BattleCli, e para o BattlegroundCatalog, usado só pelo BattleRunner. Desconsiderando io e BattleRunner, a cobertura de linhas do restante do código é de 83,2% (402 de 483 linhas).